Lecture Notes: Model Calibration

CIC1205 - Machine Learning

Prof. Eduardo Bezerra

ebezerra@cefet-rj.br

April 9, 2026

0 Contents

1 Introduction **2**
1.1 Motivation . . . 2
1.2 Preliminary Concept: Data Binning . . . 3

2 What Is Model Calibration? **3**

3 Calibration Curves **5**
3.1 Construction . . . 5
3.2 Worked Example . . . 5
3.3 Interpretation . . . 7
3.4 Worked Examples . . . 8
3.4.1 Example 1 . . . 8
3.4.2 Example 2 . . . 9

4 Calibration Metrics **10**
4.1 Expected Calibration Error (ECE) . . . 10
4.2 Brier Score . . . 12
4.3 Negative Log-Likelihood (Log Loss) . . . 13
4.4 Summary of Calibration Metrics . . . 13

5 The Calibration Pipeline **14**
5.1 The calibration dataset . . . 14
5.2 Steps of the Pipeline . . . 15
5.2.1 Step 1: Train the Base Model . . . 16
5.2.2 Step 2: Fit a Calibration Model . . . 16
5.2.3 Step 3: Evaluate the Base Model on the Test Set . . . 17
5.2.4 Step 4: Evaluate the Calibrated Model on the Test Set . . . 18
5.2.5 Step 5: Deploy the Calibrated Model . . . 18
5.3 Calibration methods . . . 18
5.3.1 Platt Scaling (Sigmoid Calibration) . . . 19
5.3.2 Isotonic Regression . . . 19
5.3.3 Temperature Scaling . . . 19
5.3.4 Summary . . . 20

6 Using `CalibratedClassifierCV` in Scikit-Learn **20**
6.1 Approach 1: Pre-fitted Base Model (cv='prefit') . . . 20
6.2 Approach 2: Cross-Validated Calibration (cv=k) . . . 21
6.3 Example code . . . 22

7 When to Apply Model Calibration **22**

8 Summary **24**

1 Introduction

1.1 Motivation

The metrics covered in the *Model Evaluation* lecture notes (accuracy, precision, recall, AUC-ROC) measure a model's ability to *rank* or *classify* examples correctly, but they say little about whether the *numerical values* of the predicted probabilities can be taken at face value.

In many real-world applications the raw probability output of a classifier is used directly: a weather forecast says "70 % chance of rain"; a medical screening tool flags patients with a "risk score of 0.85"; a credit-scoring system assigns a "default probability of 0.03". In all these cases, the question is not merely *which* example receives the higher score, but whether the score itself is numerically trustworthy, that is, whether it can be interpreted as a genuine probability.

To make this concrete: suppose a model outputs $\hat{p} = 0.80$ for 100 patients. If the model is well-calibrated, we would expect approximately 80 of those patients to actually have the disease. But if only 60 of them are truly sick, the model is not necessarily misclassifying anyone; it may still correctly rank sicker patients above healthier ones. Yet, its score of 0.80 cannot be taken at face value as a probability. This gap between the predicted number and the observed number is exactly what calibration is designed to detect and correct.

More broadly, a model that outputs $\hat{p} = 0.80$ for a set of examples is effectively saying: "I am 80 % confident that these examples are positive." If, in practice, only 60 % of them actually are positive, the model is **overconfident**: its scores are systematically inflated relative to reality. The opposite problem also exists: a model that assigns $\hat{p} = 0.40$ to examples that are positive 60 % of the time is being **underconfident**; its scores consistently understate the true likelihood. In both cases, the problem is not with the *ranking* of examples (the model may still correctly order which examples are more or less likely to be positive), but with the *numerical value* of the probability itself, which can no longer be interpreted at face value. This distinction matters in practice: a risk-management system that uses $\hat{p}$ directly to make decisions will operate at the wrong threshold; a model deployed in fairness-sensitive settings may treat different groups asymmetrically; and practitioners who trust the numbers may be systematically misled.

Model calibration is the procedure by which we detect and correct this mismatch between predicted probabilities and observed frequencies. These notes describe what calibration means formally, how to diagnose miscalibration visually and numerically, and what methods are available to correct it.

Prerequisites

These notes assume familiarity with the confusion matrix, precision, recall, AUC-ROC, and the concept of a decision threshold α, all covered in the *Model Evaluation* and *Class Imbalance* lecture notes. The idea of a held-out calibration set is related to the train/validation/test split framework discussed in *Model Selection*.

Relationship to the Class Imbalance notes

The *Class Imbalance* lecture notes mention calibration in passing: a perfectly calibrated model on a 0.1 % fraud dataset will assign most transactions scores close to zero, which is correct behaviour, not a flaw. The current notes formalize that remark and extend it into a full treatment of how to measure and improve calibration.

1.2 Preliminary Concept: Data Binning

Several calibration diagnostics and metrics depend on grouping continuous predicted probabilities into intervals called **bins**. Think of a bin as a "bucket" that collects all predictions falling within a given range, so that summary statistics (mean predicted probability, observed fraction of positives, that is, the empirical probability of the positive class within that group) can be computed for each group as a whole.

For example, if predictions range from 0 to 1 and we use five equally spaced bins:

$$B_1 = [0.0,\, 0.2), \quad B_2 = [0.2,\, 0.4), \quad B_3 = [0.4,\, 0.6), \quad B_4 = [0.6,\, 0.8), \quad B_5 = [0.8,\, 1.0].$$

Each prediction is assigned to exactly one bin, and the bin becomes the unit of analysis for calibration diagnostics. For instance, all predictions between 0.60 and 0.70 land in bin B_4, regardless of their exact value.

The choice of bin width (or, equivalently, the number of bins M) involves a bias-variance trade-off: too few bins obscure local miscalibration; too many bins yield noisy estimates because each bin contains few examples. In practice, $M = 10$ or $M = 15$ is a common default for datasets of moderate size.

2 What Is Model Calibration?

Model Calibration

Model calibration is a procedure to **adjust** the output probabilities of a classifier so that they better reflect the true probabilities of the predicted outcomes. A model is said to be **well-calibrated** if, among all instances assigned a predicted probability $\hat{p}(\mathbf{x}) \approx q$, approximately a fraction q of them are genuinely positive.

Formally, a binary classifier is perfectly calibrated when, for every value $q \in [0, 1]$:

$$\Pr\big(Y = 1 \mid \hat{p}(\mathbf{x}) \approx q\big) \approx q.$$

In words: if the model says "this event has a 70 % chance of occurring", then among all events the model so describes, approximately 70 % should actually occur. This condition is approximated in practice by grouping predictions into bins around q and comparing the average predicted probability with the empirical fraction of positives within that bin, which is exactly what the binning procedure (Section 1.2) implements.

Technical remark on the formal definition

A common statement of perfect calibration uses exact equality: $\Pr(Y = 1 \mid \hat{p}(\mathbf{x}) = q) = q$. However, for continuous-valued outputs, the event $\hat{p}(\mathbf{x}) = q$ has probability zero, making this equation technically ill-defined. The operationally correct reading is the one above: among instances whose predicted probability falls in a small neighbourhood of q, approximately a fraction q should be positive. This is precisely what the binning procedure (Section 1.2) approximates in practice.

Calibration and discrimination are orthogonal. A common source of confusion is equating good discrimination (high AUC-ROC) with good calibration. To see why they are independent, consider two models evaluated on the same test set:

Model	Predicted scores	True labels	AUC-ROC
Model A	0.90, 0.80, 0.70	1, 1, 0	0.75
Model B	0.62, 0.55, 0.52	1, 1, 0	0.75

Both models produce the same ranking (and therefore the same AUC-ROC) but their probability values differ substantially. To understand why Model A is likely *overconfident*, recall what a well-calibrated score should represent: if a model assigns a score of 0.90 to an example, then roughly 90% of examples with that score should truly be positive. In this test set, 2 out of 3 examples are positive, giving a base rate of approximately 67%. Model A assigns scores of 0.90, 0.80, and 0.70, the last of which goes to the *negative* example (true label = 0). Saying "70% chance of being positive" for something that is actually negative suggests the model is systematically inflating its probability estimates beyond what the data supports.

Model B, by contrast, assigns scores of 0.62, 0.55, and 0.52, much closer to the empirical base rate, and nearly 0.50 ("uncertain") for the negative example. Its scores are more conservative and better reflect the actual prevalence of positives in the data.

Crucially, neither model's ranking changes: both assign the highest score to the first example, the second-highest to the second, and the lowest to the third. This is why AUC-ROC, which is purely rank-based, is identical for both. The AUC captures *ordering*, not the *magnitude* of the probabilities.

It should be noted that with only three data points, it is impossible to rigorously measure calibration; the example is purely illustrative. Its purpose is to show that two models can be indistinguishable by a ranking metric while producing very different probability values.

More generally:

- A model with **high discrimination** (high AUC-ROC) ranks positives above negatives but may still be badly miscalibrated: the scores may be systematically too high or too low.
- A model with **good calibration** has trustworthy probability values but may still rank examples poorly.

In practice, both properties matter, and they should be assessed with separate metrics.

3 Calibration Curves

A **calibration curve** (also called a **reliability diagram** or **calibration plot**) is the primary visual tool for diagnosing miscalibration. It plots the mean predicted probability per bin against the observed fraction of positives (i.e., the empirical positive rate) in that bin.

3.1 Construction

Given a set of predictions $\{(\hat{p}_i, y_i)\}_{i=1}^n$ on a held-out dataset, the calibration curve is built as follows.

Step 1: Define the bins. Divide the $[0, 1]$ interval into M bins, typically of equal width:

$$B_m = \big[(m-1)/M,\ m/M\big), \quad m = 1, \ldots, M,$$

with the last bin closed on the right. Each prediction $\hat{p}_i$ is assigned to the bin B_m that contains it.

Step 2: Compute bin-level statistics. For each bin B_m containing $|B_m|$ predictions, compute:

$$\mathrm{conf}(B_m) = \frac{1}{|B_m|} \sum_{i \in B_m} \hat{p}_i \qquad \text{(mean predicted probability, “model confidence”)}$$

$$\mathrm{acc}(B_m) = \frac{1}{|B_m|} \sum_{i \in B_m} y_i \qquad \text{(observed fraction of positives in the bin)}$$

Step 3: Plot. Plot the pairs $\big(\mathrm{conf}(B_m), \mathrm{acc}(B_m)\big)$ for $m = 1, \ldots, M$. Also draw the diagonal $y = x$, which represents perfect calibration.

3.2 Worked Example

Consider the following ten predictions from a binary classifier presented in Table 1. Each half of the table lists individual prediction outcomes. The **Instance** column is simply an identifier for each observation. The **Prob.** column contains the predicted probability $\hat{p}_i$ output by the base model for the positive class, and the **Label** column contains the corresponding true binary outcome $y_i \in \{0, 1\}$. The two halves are presented side by side purely for compactness; together they form a single dataset of 24 instances that will be grouped into bins to compute the quantities $\mathrm{conf}(B_m)$ and $\mathrm{acc}(B_m)$ used in the reliability diagram.

(a) Instances 1–12			(b) Instances 13–24		
Inst.	**Prob.**	**Label**	**Inst.**	**Prob.**	**Label**
1	0.10	0	13	0.52	1
2	0.12	0	14	0.58	0
3	0.15	1	15	0.60	1
4	0.18	0	16	0.65	0
5	0.22	1	17	0.68	1
6	0.28	0	18	0.72	0
7	0.30	1	19	0.75	1
8	0.35	1	20	0.78	0
9	0.40	0	21	0.82	1
10	0.45	1	22	0.88	1
11	0.48	0	23	0.92	0
12	0.50	1	24	0.95	1

Table 1: Toy dataset (24 instances) used to construct the reliability diagram. Each bin contains multiple examples, allowing non-extreme empirical accuracies.

Let us used five bins, each of width 0.2: B_1: $[0.0, 0.2)$, B_2: $[0.2, 0.4)$, B_3: $[0.4, 0.6)$, B_4: $[0.6, 0.8)$, and B_5: $[0.8, 1.0]$. The bin-level summary is presented in Table 2.

Bin	**Predictions**	$\lvert B_m \rvert$	$\mathrm{conf}(B_m)$	$\mathrm{acc}(B_m)$	**Interpretation**
B_1	0.10, 0.12, 0.15, 0.18	4	0.137	0.25	Underconfident
B_2	0.22, 0.28, 0.30, 0.35	4	0.287	0.75	Underconfident
B_3	0.40, 0.45, 0.48, 0.50, 0.52, 0.58	6	0.488	0.50	Well-calibrated
B_4	0.60, 0.65, 0.68, 0.72, 0.75, 0.78	6	0.697	0.50	Overconfident
B_5	0.82, 0.88, 0.92, 0.95	4	0.893	0.75	Overconfident

Table 2: Bin-level summary for the toy dataset with 24 instances. Each bin B_m groups predictions by confidence, reporting the number of instances $|B_m|$, the mean predicted probability $\mathrm{conf}(B_m)$, and the observed fraction of positives $\mathrm{acc}(B_m)$. Unlike the smaller example, each bin now contains multiple observations with mixed labels, yielding non-extreme empirical accuracies and a more informative reliability diagram.

Figure 1 shows the resulting calibration curve. The points that lie above the diagonal correspond to bins where the model is *underconfident* (the observed fraction of positives exceeds the predicted probability); points below the diagonal correspond to *overconfidence*.

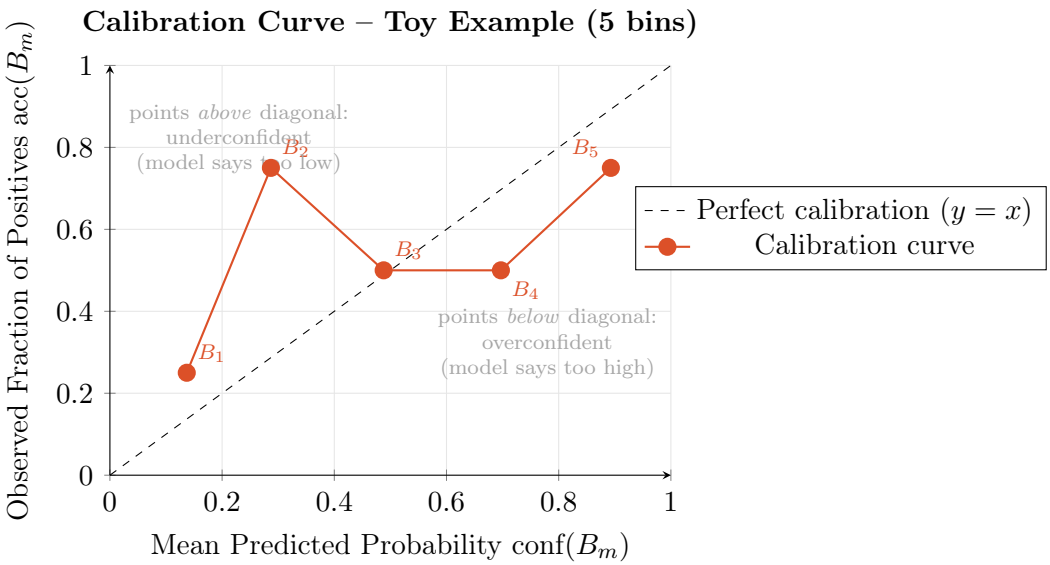


Figure 1: Calibration curve for the 24-instance toy example. Each point corresponds to one bin; the diagonal $y = x$ represents perfect calibration. Points below the diagonal indicate overconfidence (the model assigns probabilities that are too high relative to the observed fraction of positives), whereas points above the diagonal indicate underconfidence.

3.3 Interpretation

Let B_m be any bin. Then:

$$\begin{cases} \text{conf}(B_m) > \text{acc}(B_m) & \text{model is } \textbf{overconfident} \text{ in } B_m \\ & \text{(curve } \textit{below} \text{ diagonal: model overestimates probability)} \\ \text{conf}(B_m) < \text{acc}(B_m) & \text{model is } \textbf{underconfident} \text{ in } B_m \\ & \text{(curve } \textit{above} \text{ diagonal: model underestimates probability)} \\ \text{conf}(B_m) \approx \text{acc}(B_m) & \text{model is } \textbf{well-calibrated} \text{ in } B_m \end{cases}$$

A well-calibrated model has all points lying close to the diagonal $y = x$.

Quick reference – direction convention (easy to confuse!)

- **Curve BELOW the diagonal ⇒ overconfident.** The model assigns higher probabilities than the observed positive rate warrants: $\text{conf}(B_m) > \text{acc}(B_m)$.
- **Curve ABOVE the diagonal ⇒ underconfident.** The model assigns lower probabilities than the observed positive rate warrants: $\text{conf}(B_m) < \text{acc}(B_m)$.

This direction is counterintuitive to many students ("the model is too high, so shouldn't the curve be above?"). Remember: the x-axis is the model's predicted probability and the y-axis is the reality. If the model is too high (conf > acc), reality is *below* the model's claim, hence the point falls *below* the diagonal.

Common miscalibration patterns. Different model families tend to produce characteristic shapes on the calibration curve.

- **Unpruned Decision Trees** are typically *overconfident*: their probability estimates cluster near 0 and 1, pulling curves toward the axes. This is because the tree structure pushes predictions toward pure leaves.
- **Random Forests** are typically *underconfident*: averaging the outputs of many trees compresses predicted probabilities toward 0.5, producing a calibration curve that tends to lie *above* the diagonal [Niculescu-Mizil and Caruana, 2005]. This is the opposite pattern to that of individual unpruned trees.
- **Gradient Boosted Trees** can exhibit overconfidence, particularly when the number of trees is large and regularisation is insufficient; the degree depends on the specific implementation and hyperparameter choices.
- **Naive Bayes** is often *underconfident*: the feature-independence assumption tends to push probabilities toward 0.5, producing an S-shaped curve that lies above the diagonal for low scores and below for high scores.
- **Logistic regression** is generally well-calibrated for the class of problems to which it is well-specified, because the log-loss it minimizes is a proper scoring rule.
- **Deep neural networks** tend to be *overconfident*, especially when trained with cross-entropy loss without regularization; recent work has shown that modern networks are significantly more miscalibrated than their predecessors [Guo et al., 2017].

Figure 2 illustrates the three qualitative patterns that arise in practice. Each panel shares the same structure: the x-axis shows the model's mean predicted probability within each bin ($\text{conf}(B_m)$), the y-axis shows the observed fraction of positives ($\text{acc}(B_m)$), and the dashed diagonal represents perfect calibration, i.e., the line along which predicted probability equals observed frequency. The coloured points are the bin-level summaries, and their position relative to the diagonal reveals the calibration behaviour of the model. In the **well-calibrated** panel (left), the points track the diagonal closely: a predicted probability of, say, 0.70 corresponds to roughly 71% of examples in that bin being positive. In the **overconfident** panel (centre), the curve falls *below* the diagonal throughout. This means that for every bin the model's confidence exceeds the true positive rate, predicting 0.70, for instance, when only about 52% of those examples are actually positive. The model is, in effect, too assertive: it assigns high probabilities to events that occur less often than it claims. In the **underconfident** panel (right), the curve lies *above* the diagonal: the true positive rate consistently exceeds the predicted probability. Here a prediction of 0.30 is associated with nearly 48% positives, so the model is more conservative than the data warrants, hedging its estimates toward the centre even when the evidence is strong.

3.4 Worked Examples

3.4.1 Example 1

A calibration curve for a new model is plotted and you observe that most of its points fall *below* the diagonal. Which of the following best describes the model?

(a) The model is underconfident: it assigns probabilities that are too low.

(b) The model is overconfident: it assigns probabilities that are too high.

(c) The model is well-calibrated.

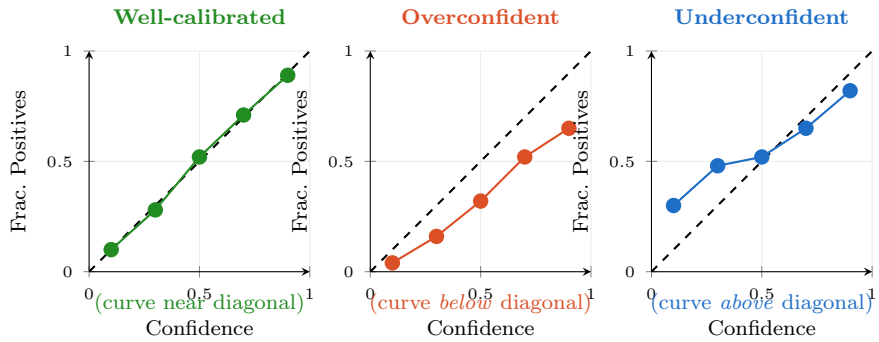


Figure 2: Three characteristic calibration patterns. A well-calibrated model (left) has its curve close to the diagonal. An overconfident model (centre) lies *below* the diagonal: its predicted probabilities are higher than the observed fraction of positives warrants (model overestimates probability). An underconfident model (right) lies *above* the diagonal: it is more conservative than the data supports (model underestimates probability).

(d) The model has poor discrimination (low AUC-ROC).

Solution *(b). Points below the diagonal mean conf$(B_m) >$ acc(B_m), so the model predicts higher probabilities than the observed positive rate warrants. Note that (d) is not implied: poor calibration and poor discrimination are independent properties.*

3.4.2 Example 2

A random forest is trained on a medical screening task. On a held-out validation set of 1,000 patients, you build a calibration curve with 10 bins.

1. Bin B_8 (covering $[0.7, 0.8)$) contains 40 patients with mean predicted probability 0.74, of whom 18 are actually sick. Is the model overconfident or underconfident in this bin?
2. Across all bins, the curve lies consistently below the diagonal. What does this imply about the model's probability estimates?
3. A colleague argues: "our model has AUC-ROC $= 0.90$, so calibration doesn't matter." Give one concrete scenario in which good discrimination but poor calibration would be harmful.
4. *(Sketch)* Roughly sketch the shape of a calibration curve for an overconfident model. On your sketch, mark where the curve lies relative to the diagonal and label one point as an example.
5. Would changing the classification threshold (e.g., from 0.5 to 0.3) fix the calibration problem revealed in item 2? Explain why or why not.

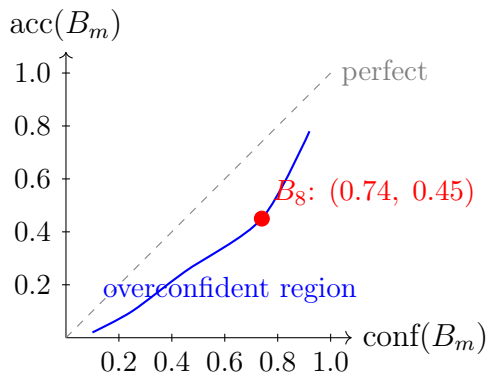
Solution

1. $\text{conf}(B_8) = 0.74$ and $\text{acc}(B_8) = 18/40 = 0.45$. Since $\text{conf}(B_8) > \text{acc}(B_8)$, the model is *overconfident* in this bin: it predicts a $\sim$74% probability of disease for patients who, in reality, have a positive rate of only 45%.

2. A curve that lies consistently below the diagonal implies $\text{conf}(B_m) > \text{acc}(B_m)$ across virtually all bins, meaning the model *systematically overestimates* probabilities. For any score range, the true positive rate is lower than the model claims.

3. A concrete scenario: suppose the model's score is used to communicate risk to the patient or to determine the intensity of treatment. For example, referring only patients with a predicted probability above 0.80 for surgery. If the model is overconfident, patients with a score of 0.85 may have a true risk of only 0.50, leading to unnecessary and potentially harmful interventions. A high AUC guarantees that the sickest patients rank at the top, but it does not guarantee that the score itself is interpretable as a true probability.

4. The calibration curve of an overconfident model lies systematically below the diagonal: the points are "pulled" to the right (high conf) while the corresponding acc remains low. The sketch below illustrates this behaviour; the marked point corresponds to bin B_8 from item 1.



5. No. Changing the classification threshold (e.g., from 0.5 to 0.3) shifts *where the decision boundary is drawn* between the classes, but does not alter the probability scores themselves. Calibration is a property of the raw scores (i.e., the relationship between $\text{conf}(B_m)$ and $\text{acc}(B_m)$ within each bin) and remains unchanged regardless of the chosen threshold. To correct the calibration problem identified in item 2, a post-processing method such as *Platt scaling* or *isotonic regression* would be required; these techniques transform the scores to align them with the observed frequencies.

4 Calibration Metrics

Visual inspection of the calibration curve reveals the *pattern* of miscalibration but does not provide a single number with which to compare models quantitatively or track improvement over time. Three complementary scalar metrics are commonly used for this purpose.

4.1 Expected Calibration Error (ECE)

The **Expected Calibration Error** measures the average absolute gap between confidence and observed fraction of positives, weighted by bin size:

ECE definition

$$\mathrm{ECE} = \sum_{m=1}^{M} \frac{|B_m|}{n} \left| \mathrm{acc}(B_m) - \mathrm{conf}(B_m) \right|$$

where n is the total number of predictions and $|B_m|$ is the number of predictions in bin B_m.

ECE is essentially a weighted mean of the per-bin calibration errors, where larger bins receive proportionally more weight. A perfectly calibrated model achieves ECE $= 0$; lower values indicate better calibration.

ECE depends on the binning strategy; interpret with caution

ECE is *not* a uniquely defined metric: its value depends on the number of bins M and on the binning strategy (equal-width intervals vs. equal-frequency quantile bins). Different binning schemes can yield substantially different ECE values for the same model. Therefore, ECE is not a canonical metric and should always be reported together with the binning configuration used. Comparisons of ECE across studies or models are only meaningful when the exact same binning scheme is applied consistently. In practice, always report the number of bins and strategy alongside the ECE value.

A related variant, the **Maximum Calibration Error** (MCE), reports the worst-case gap across all bins:

$$\mathrm{MCE} = \max_{m=1,\ldots,M} \left| \mathrm{acc}(B_m) - \mathrm{conf}(B_m) \right|.$$

MCE is useful when the worst-case miscalibration is more important than the average (e.g., in safety-critical applications).

Numerical example. Using the five-bin toy example from Section 3 and $n = 10$, we get the results presented in Table 3.

m	$\lvert B_m \rvert$	$\mathrm{conf}(B_m)$	$\mathrm{acc}(B_m)$	$\frac{\lvert B_m \rvert}{n} \lvert \mathrm{acc} - \mathrm{conf} \rvert$
1	4	0.137	0.25	$\frac{4}{24} \times 0.113 = 0.019$
2	4	0.287	0.75	$\frac{4}{24} \times 0.463 = 0.077$
3	6	0.488	0.50	$\frac{6}{24} \times 0.012 = 0.003$
4	6	0.697	0.50	$\frac{6}{24} \times 0.197 = 0.049$
5	4	0.893	0.75	$\frac{4}{24} \times 0.143 = 0.024$
			ECE	**0.172**

Table 3: Computation of the Expected Calibration Error (ECE) for the 24-instance toy example. For each bin B_m, the contribution is given by the bin weight $\frac{|B_m|}{n}$ multiplied by the absolute difference between the observed fraction of positives $\mathrm{acc}(B_m)$ and the mean predicted probability $\mathrm{conf}(B_m)$.

This is a high ECE, which reflects the noisy calibration of the toy example (only 10 data points). In general, lower ECE is better. What constitutes an acceptable ECE depends on the application domain, the base rate of the positive class, and the costs of miscalibration. There is no universal threshold, and thresholds cited in the literature (such as 0.05) should be treated as rough guidelines rather than hard rules.[1]

4.2 Brier Score

The **Brier Score** measures the mean squared error between predicted probabilities and true binary outcomes:

Brier Score definition

$$\mathrm{BS} = \frac{1}{n}\sum_{i=1}^{n}\big(\hat{p}(\mathbf{x}_i) - y_i\big)^2$$

The Brier Score ranges from 0 (perfect predictions) to 1 (worst possible). It is a **strictly proper scoring rule**: it is minimized when $\hat{p}(\mathbf{x}_i)$ equals the true conditional probability $\Pr(Y = 1 \mid \mathbf{x}_i)$ for every example. Proper scoring rules provide an incentive to report honest probabilities.

An important decomposition relates the Brier Score to three interpretable components [Murphy, 1973]:

$$\mathrm{BS} = \underbrace{\text{Reliability}}_{\text{calibration error}} - \underbrace{\text{Resolution}}_{\text{discrimination power}} + \underbrace{\text{Uncertainty}}_{\text{inherent difficulty}}.$$

Each term has a concrete meaning: **Reliability** quantifies how far the model's predicted probabilities deviate from the observed positive rates (it is minimized by good calibration); **Resolution** measures how much the model's predictions vary across instances; a model that always predicts the base rate has zero resolution; **Uncertainty** reflects the inherent difficulty of the task, captured by the overall positive rate $\bar{y}(1-\bar{y})$, and is independent of the model.

The term **Reliability** in the Brier–Murphy decomposition is **not the same as ECE**. Formally:

$$\text{Reliability} = \frac{1}{n}\sum_{m=1}^{M}|B_m|\,\big(\mathrm{acc}(B_m) - \mathrm{conf}(B_m)\big)^{\mathbf{2}},$$

where the deviation is *squared*, whereas ECE uses the *absolute value*:

$$\mathrm{ECE} = \sum_{m=1}^{M}\frac{|B_m|}{n}\,\big|\,\mathrm{acc}(B_m) - \mathrm{conf}(B_m)\big|.$$

[1] There is no universally accepted threshold for ECE. The value of 0.05 is often cited as a reference for good calibration [Guo et al., 2017], but should be treated as a rough guideline rather than a hard rule. The appropriate threshold depends on the application domain, the base rate of the positive class, and the costs of miscalibration.

Both measure calibration error, but they are mathematically distinct quantities and are not directly comparable. When reading research papers, be careful not to conflate them.

Note the sign of each term carefully: the negative sign on Resolution means that *higher* resolution *reduces* the Brier Score (i.e., improves the model), whereas higher Reliability error *increases* it. A model therefore improves its Brier Score either by improving calibration (reducing Reliability) or by increasing its ability to distinguish positives from negatives (increasing Resolution).

4.3 Negative Log-Likelihood (Log Loss)

The **Negative Log-Likelihood**, also called **log loss**, is the cross-entropy between the true labels and the predicted probabilities:

Log Loss definition

$$\mathrm{NLL} = -\frac{1}{n}\sum_{i=1}^{n}\Big[y_i \log \hat{p}(\mathbf{x}_i) + (1-y_i)\log\big(1-\hat{p}(\mathbf{x}_i)\big)\Big]$$

Like the Brier Score, log loss is a strictly proper scoring rule. It penalizes overconfident wrong predictions very heavily: if the model assigns $\hat{p} \approx 1$ to an example whose true label is 0, the term $\log(1-\hat{p})$ approaches $-\infty$. This asymmetric penalty makes log loss particularly sensitive to miscalibration on confident predictions.

Log loss as a training objective

Many classifiers (logistic regression, neural networks trained with cross-entropy loss) already minimize log loss during training. Such models tend to have good calibration on the training distribution, but can still be miscalibrated on out-of-distribution data or after distribution shift.

4.4 Summary of Calibration Metrics

Metric	Interpretation	Notes
ECE	Average absolute gap between confidence and observed positive rate per bin	Depends on binning; lower is better
MCE	Worst-case gap across bins	Useful for safety-critical applications
Brier Score	Mean squared error of probability predictions	Strictly proper; Reliability term uses *squared* error (distinct from ECE)
Log Loss (NLL)	Cross-entropy between predictions and true labels	Strictly proper; very sensitive to overconfident errors

When to use which metric

ECE is the most common metric for reporting calibration because it is easy to interpret and directly linked to the calibration curve. Brier Score and log loss are useful when calibration is being optimized as a training objective or when comparing models across different evaluation frameworks. In practice, report ECE alongside AUC-ROC (or PR-AUC for imbalanced data) to cover both calibration and discrimination.

Entry Ticket 1

A probabilistic classifier is evaluated on a test set of 5 examples with predictions and labels:

i	$\hat{p}_i$	y_i
1	0.90	1
2	0.80	0
3	0.60	1
4	0.30	0
5	0.10	0

1. Compute the Brier Score for this model.
2. Instance 2 has $\hat{p}_2 = 0.80$ but $y_2 = 0$. Compute its contribution to the log loss. What happens to this contribution if $\hat{p}_2$ is increased to 0.99? What does this reveal about log loss's treatment of overconfident errors?
3. Suppose you use only 2 bins: $[0, 0.5)$ and $[0.5, 1]$. Compute ECE for these 5 examples.

5 The Calibration Pipeline

So far, we have discussed how to diagnose and measure miscalibration through calibration curves and scalar metrics. We now turn to the practical question: how to correct miscalibration in a real deployed system.

Calibration is always applied *post hoc*: first a base model is trained to maximize discrimination; then its probability outputs are adjusted by a separate calibration model fitted on held-out data.

5.1 The calibration dataset

Befor running a calibration pipeline, the data must be divided into four non-overlapping subsets:

- **Training set**: used exclusively to train the base model.
- **Validation set**: used for model selection, that is, for comparing candidate models and tuning hyperparameters, as covered in the *Model Selection* notes.
- **Calibration set**: reserved for fitting the calibration model. It must be strictly unseen by the base model; using the same data for both training and calibration would introduce data leakage and yield overoptimistic results.

- **Test set**: touched only once, at the very end, to compute calibration and discrimination metrics on the complete calibrated system. Because neither the base model nor the calibration model has ever seen this data, the resulting metrics provide unbiased estimates of the generalisation performance expected on genuinely new data.

A typical four-way split is 60 % training, 10 % validation, 10 % calibration, and 20 % test, though the optimal proportions depend on dataset size and the complexity of the calibration method. Table 4 summarises the role of each split and relates it to the terminology used in the *Model Selection* notes.

Split	Role in Model Selection	Role in Calibration
Train	Train candidate models	Train the base model
Validation	Compare models, tune hyperparams	Encapsulated inside `select_and_train`
Calibration	(not used)	Fit the calibration model
Test	Estimate generalisation error	Estimate generalisation error

Table 4: Role of each data split in Model Selection vs. Calibration. The test set serves the same purpose in both contexts.

Why data leakage is especially harmful in calibration

Data leakage is a general concern in machine learning, but it has a particularly insidious form in calibration. When a model is evaluated on examples it has seen during training, its probability outputs are artificially optimistic: the model has effectively "memorised" those examples, so its scores no longer reflect genuine uncertainty. Fitting the calibration model on such predictions yields a calibration function that learns to correct a bias that does not exist on new data, and therefore miscalibrates the model on any real deployment. The calibration set must be strictly unseen by the base model.

5.2 Steps of the Pipeline

The pipeline comprises five steps, illustrated in Figure 3. A prior data split is assumed, producing three disjoint subsets: a development set (dev set), a calibration set, and a test set.

The dev set contains both training and validation data and is used during model development. The calibration and test sets are held out and used only in subsequent stages.

Note on the pipeline diagram

The validation set does not appear explicitly in the Step 1→2→3→4→5 flow because it is *encapsulated within Step 1*: the model development step internally performs training and validation (e.g., hyperparameter tuning) using the dev set. The diagram abstracts this sub-problem into a single step for clarity.

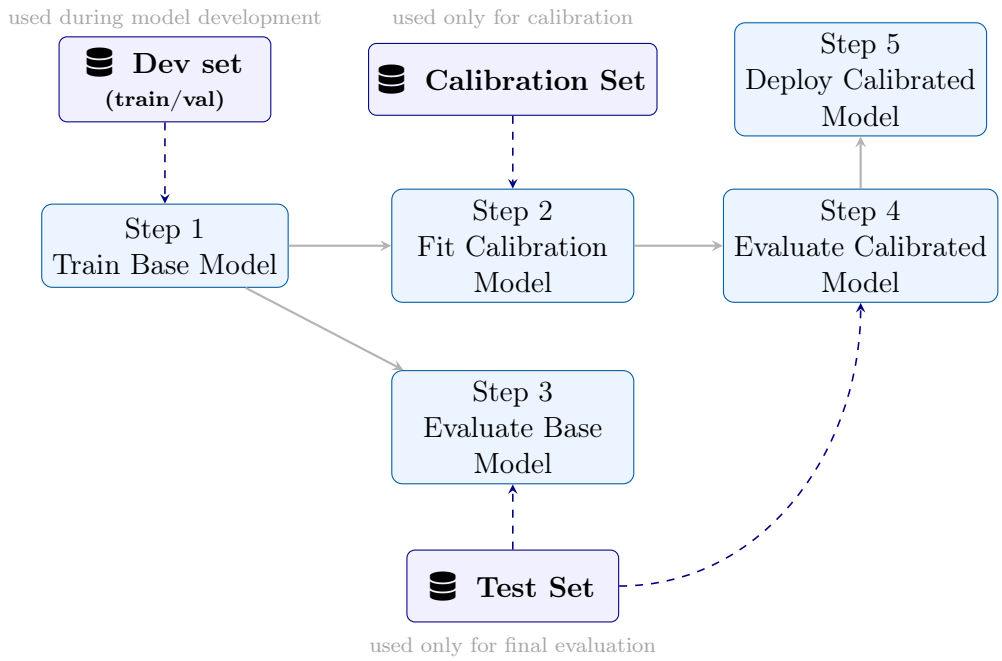


Figure 3: A calibration pipeline with datasets represented explicitly as inputs to the relevant steps. A prior data split is assumed, producing three disjoint subsets: a dev set, a calibration set, and a test set. The dev set is used to train and select the base model, the calibration set is used only to fit the calibration model after the base model has been fixed, and the test set is used only for final evaluation of both the uncalibrated and calibrated models.

5.2.1 Step 1: Train the Base Model

Using the dev set (which includes both training and validation splits), apply the model selection procedure covered in the *Model Selection* notes: train candidate models, evaluate them on validation data, tune hyperparameters, and return the best fitted model.

This step is typically implemented as a function such as `select_and_train`, which takes both training and validation data as input and outputs a fully specified model.

The classifier must output *probability estimates*, not just class labels. Examples include logistic regression, random forests, gradient boosting, and neural networks (all of which provide `predict_proba` in scikit-learn).

At this stage, the model's probabilities need not be well-calibrated; discrimination (AUC-ROC or PR-AUC) is the primary objective. Calibration is addressed in the next step.

5.2.2 Step 2: Fit a Calibration Model

Using the calibration set, fit a *calibration model* that maps the base model's raw probability outputs to calibrated probabilities.

The calibration model treats the base model's scores as a single input feature and the original labels as targets. This reduces the problem to a univariate binary classification task: learning a function $f : [0, 1] \to [0, 1]$ such that $f(\hat{p})$ better reflects empirical class frequencies.

Figure 4 illustrates the structure of the calibration dataset. Each observation consists of:

- the base model's predicted probability $\hat{p}_i$ (input feature),
- the true label $y_i \in \{0, 1\}$ (target).

No original input features are used in this step. This design keeps calibration modular and model-agnostic.

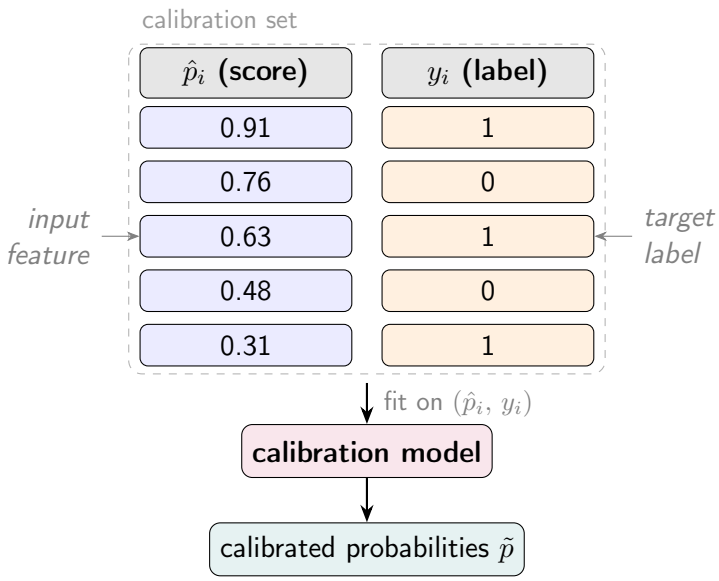


Figure 4: Structure of the calibration dataset. The base model's raw score $\hat{p}_i$ is the sole input feature; the true label y_i is the target. The calibration model learns a mapping to calibrated probabilities $\tilde{p}$.

In practice, the calibration model can be fitted using different methods, depending on the characteristics of the base model and the size of the calibration set. Common choices include parametric approaches such as Platt scaling, non-parametric methods such as isotonic regression, and specialized techniques such as temperature scaling for neural networks. These methods differ in flexibility, data requirements, and assumptions about the shape of the miscalibration. A detailed discussion of these approaches is provided in Section 5.3.

5.2.3 Step 3: Evaluate the Base Model on the Test Set

Before applying calibration, evaluate the base model on the test set. This provides a reference point for comparison.

Use both:

- **Discrimination metrics**: AUC-ROC, PR-AUC, F_β.
- **Calibration metrics**: ECE, Brier Score, NLL.

This step answers the question: *how good are the raw probabilities produced by the base model?*

5.2.4 Step 4: Evaluate the Calibrated Model on the Test Set

After fitting the calibration model, evaluate the full system (base model followed by calibration model) on the **same test set**.

Using the same dataset ensures a fair comparison between:

- the uncalibrated base model (Step 3),
- the calibrated model (this step).

Again, use both calibration and discrimination metrics.

The effect of calibration on AUC-ROC depends on the method used:

- Platt Scaling and Temperature Scaling are strictly monotone transformations and preserve ranking exactly. Therefore, AUC-ROC remains unchanged.
- Isotonic Regression is monotone but may introduce flat regions (ties), which can slightly reduce AUC-ROC.

The goal is to improve calibration metrics (e.g., reduce ECE) while preserving discrimination performance.

5.2.5 Step 5: Deploy the Calibrated Model

In deployment, replace the base model's raw outputs with calibrated probabilities.

Calibrated probabilities are essential whenever the probability value itself (not just ranking) is used for decision-making:

- Threshold-based decisions with asymmetric costs.
- Expected utility maximization.
- Risk communication (e.g., "15% probability of disease").
- Model aggregation (ensemble averaging requires calibrated inputs).

Entry Ticket 2

A team trains a neural network for disease diagnosis and wants to apply calibration.

1. The team proposes fitting the calibration model on the same data used for training the neural network. Identify the methodological error and explain why it leads to overly optimistic results.
2. Which calibration method would you recommend first for a neural network, and why? What is its key hyperparameter?
3. After calibration, AUC-ROC remains at 0.88 while ECE drops from 0.12 to 0.03. What does this indicate about the effect of calibration?

5.3 Calibration methods

Three calibration methods are widely used. They are discussed next.

5.3.1 Platt Scaling (Sigmoid Calibration)

Platt scaling [Platt, 1999] fits a logistic regression on the base model's raw scores. Let $s(\mathbf{x})$ denote a real-valued score produced by the base model. This score may be a logit (pre-sigmoid output of a neural network), a decision function value (as in SVMs), or a raw probability estimate, depending on the model. Platt scaling then fits the transformation:

$$\hat{p}_{\text{cal}}(\mathbf{x}) = \sigma\big(a \cdot s(\mathbf{x}) + b\big) = \frac{1}{1 + e^{-(a\, s(\mathbf{x}) + b)}},$$

where the parameters a, b are estimated by maximum likelihood on the calibration set.

Platt scaling is parametric: it assumes a sigmoid relationship between the base model's scores and the true probabilities, which works well when $s(\mathbf{x})$ is approximately linear in the log-odds of the outcome (e.g., for SVMs or models with slightly skewed output distributions). It requires relatively few calibration examples (a few hundred suffice).

5.3.2 Isotonic Regression

Isotonic regression [Zadrozny and Elkan, 2002] fits a non-parametric piecewise constant, monotonically non-decreasing function:

$$\hat{p}_{\text{cal}}(\mathbf{x}) = f\big(s(\mathbf{x})\big),$$

where f is estimated by the pool adjacent violators (PAV) algorithm, which finds the isotonic function that minimizes the squared error on the calibration set.

Isotonic regression is more flexible than Platt scaling: it can correct arbitrary monotone miscalibration, not only sigmoid-shaped distortions. However, it requires a larger calibration set (typically several thousand examples) to avoid overfitting. When the calibration set is small, the PAV algorithm may overfit, learning idiosyncrasies of the calibration data that do not generalize to new examples; this can degrade calibration on the test set and indirectly affect discrimination metrics. Even with sufficient data, isotonic regression can produce a staircase function with flat regions, meaning that different raw scores may be mapped to the same calibrated probability, potentially introducing ties that can slightly reduce AUC-ROC compared to the uncalibrated model. In practice this effect is usually small, but both risks are worth being aware of when choosing between methods.

5.3.3 Temperature Scaling

Temperature scaling [Guo et al., 2017] is a simple but effective technique for calibrating models whose architecture exposes a logit layer. Let $z(\mathbf{x})$ denote the logit (pre-sigmoid output) of the base model, so that the original predicted probability is $\hat{p}(\mathbf{x}) = \sigma(z(\mathbf{x}))$. Temperature scaling divides this logit by a single scalar parameter $T > 0$ before applying the activation:

$$\hat{p}_{\text{cal}}(\mathbf{x}) = \sigma\left(\frac{z(\mathbf{x})}{T}\right).$$

When $T > 1$, the logit is shrunk toward zero, smoothing the output distribution and lowering confidences; this corrects overconfidence. When $T < 1$, confidences are sharpened, which is useful for underconfident models. The parameter T is estimated by minimizing the NLL on the calibration set.

For multiclass problems, the same idea generalizes naturally: the entire logit vector is divided by T before applying the softmax, leaving the predicted class (the argmax) unchanged.

Temperature scaling is particularly popular for calibrating modern deep neural networks because (i) it does not change the model's predictions (the argmax of the softmax is invariant to T) and (ii) it requires optimizing only one parameter, making it robust even when the calibration set is moderate in size.

5.3.4 Summary

Table 5 summarizes the main characteristics of the most commonly used calibration methods. The choice of method depends primarily on the amount of available calibration data and on the expected shape of the miscalibration. Parametric approaches such as Platt scaling are appropriate when data are limited and the distortion is approximately sigmoid-shaped, whereas non-parametric methods such as isotonic regression offer greater flexibility at the cost of requiring larger datasets to avoid overfitting. Temperature scaling provides a particularly simple and robust alternative for neural networks, correcting mainly overconfidence while preserving the model's ranking.

Method	**When to use**	**Data needed**
Platt Scaling	Simple models; sigmoid-shaped output; limited calibration data	Small (~few hundred)
Isotonic Regression	More flexible needs; larger calibration set available	Large (~thousands)
Temperature Scaling	Neural networks; only overconfidence to correct	Moderate

Table 5: Comparison of common calibration methods. The methods differ in flexibility, assumptions about the shape of miscalibration, and the amount of calibration data required. Simpler parametric methods (e.g., Platt scaling) are suitable for small datasets, whereas more flexible approaches (e.g., isotonic regression) require larger calibration sets to avoid overfitting.

6 Using `CalibratedClassifierCV` in Scikit-Learn

Scikit-learn provides the class `CalibratedClassifierCV` to apply post-hoc calibration to any classifier that implements `predict_proba`. The class handles both Platt scaling (`method='sigmoid'`) and isotonic regression (`method='isotonic'`).

There are two usage patterns, differing in how the base model is trained and how the train/calibration split is managed.

6.1 Approach 1: Pre-fitted Base Model (cv='prefit')

In this approach, the base model is trained outside of `CalibratedClassifierCV` on a separate training set, and the calibration model is fitted on a disjoint calibration set.

1. Obtain two disjoint datasets: `(X_train, y_train)` for the base model and `(X_calib, y_calib)` for calibration.
2. Train the base model: `model.fit(X_train, y_train)`.
3. Create a `CalibratedClassifierCV` instance with `cv='prefit'` (indicating that the base model is already trained): `cccv = CalibratedClassifierCV(model, cv='prefit')`.
4. Fit the calibration model: `cccv.fit(X_calib, y_calib)`.

When to use. This approach is preferred when the dataset is large enough to split into three parts (training, calibration, and test), or when the base model has already been trained and only calibration is needed as a post-processing step.

Common mistake with cv='prefit'

A frequent error is to call `cccv.fit(X_train, y_train)`, that is, passing the same data used to train the base model. Because the base model has already seen those examples, its `predict_proba` outputs are overoptimistic, and the calibration model will learn to correct a bias that does not exist at test time. Always pass a *held-out* calibration set to `fit` when using `cv='prefit'`.

6.2 Approach 2: Cross-Validated Calibration (cv=k)

In this approach, `CalibratedClassifierCV` handles both training and calibration via k-fold cross-validation. For each fold, the base model is trained on $k-1$ folds and the calibration model is fitted on the held-out fold. The final model averages the calibrated probabilities across all k base models.

1. Start with an untrained model instance.
2. Create: `cccv = CalibratedClassifierCV(model, cv=5)`.
3. Fit: `cccv.fit(X, y)`.

When to use. This approach is useful when the dataset is too small to afford a dedicated calibration set, since cross-validation uses all data for both training and calibration without overlap within any fold.

Disjointness is non-negotiable

Whether using Approach 1 or Approach 2, the base model must *never* be trained on the same data used to fit the calibration model. If the base model has already seen an example, its probability output for that example is over-optimistic (the model has memorized it), and using such predictions to train the calibration model will produce artificially confident calibration fits. Scikit-learn's `cv='prefit'` enforces this by requiring that the user supply a separate calibration set; `cv=k` enforces it via cross-validation.

6.3 Example code

The code below illustrates the complete pipeline, using a four-way split (train / validation / calibration / test). Comments indicate the resulting proportions relative to the full dataset, which was omitted in the original notes.

```python
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.calibration import CalibratedClassifierCV
from sklearn.metrics import brier_score_loss, roc_auc_score

# Step 1a: 20% -> test set; 80% -> temp
X_temp, X_test, y_temp, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42
)

# Step 1b: 25% of temp (= 20% of full) -> X_rest;
#          75% of temp (= 60% of full) -> X_train
X_train, X_rest, y_train, y_rest = train_test_split(
    X_temp, y_temp, test_size=0.25, random_state=42
)

# Step 1c: 50% of X_rest (= 10% of full) -> val;
#          50% of X_rest (= 10% of full) -> cal
# Final split: 60% train | 10% val | 10% cal | 20% test
X_val, X_cal, y_val, y_cal = train_test_split(
    X_rest, y_rest, test_size=0.50, random_state=42
)

# Step 2: model selection on train+val
#         (validation split is handled inside select_and_train)
model = select_and_train(RandomForestClassifier(),
                         X_train, y_train,
                         X_val, y_val)

# Step 3: fit calibration model on the held-out cal set ONLY
calibrated_model = CalibratedClassifierCV(
    model, cv='prefit', method='sigmoid'
)
calibrated_model.fit(X_cal, y_cal)

# Step 4: evaluate on test set (never seen by base or cal model)
probs = calibrated_model.predict_proba(X_test)[:, 1]
print(brier_score_loss(y_test, probs))
print(roc_auc_score(y_test, probs))
```

7 When to Apply Model Calibration

Not all applications require model calibration. Calibration is most important when the predicted probability values (as opposed to just their ranking) drive downstream decisions. The following scenarios are particularly relevant.

Probabilistic predictions used for risk assessment. When the output of a model is communicated as a probability to a human decision-maker ("this patient has a 30 % risk of readmission"), the number must be trustworthy. Miscalibrated models may systematically overstate or understate risk, leading to systematic errors across all decisions based on that probability.

Decision-making under uncertainty. When decisions are made by comparing predicted probabilities against a threshold or expected utility (see the *Class Imbalance* notes), calibration affects the threshold that would be optimal. A perfectly calibrated model allows the optimal threshold to be derived directly from the cost structure; a miscalibrated model requires an empirical threshold search that may be unstable.

Fairness and equity. If a model is calibrated overall but miscalibrated within specific demographic subgroups, it will systematically over- or under-estimate risk for those groups. Checking calibration separately for each protected group is an important component of bias auditing.

Model ensembles. When combining predictions from multiple models (e.g., averaging probabilities), each component model should be individually calibrated. Averaging miscalibrated probabilities produces a combined model that is also miscalibrated, but in a harder-to-predict way.

Regulatory and legal compliance. In high-stakes domains such as finance, healthcare, and criminal justice, regulators may require that predicted probabilities be interpretable and defensible. Calibrated models are more auditable because their outputs can be compared directly to observed frequencies in historical data.

When calibration is less critical. If the sole goal is to maximize a threshold-independent ranking metric (AUC-ROC, PR-AUC) and the probability values themselves are never communicated or used for decisions, calibration is less important. Similarly, if the base model is logistic regression (which is generally well-calibrated for in-distribution data), an explicit calibration step may add little value.

Quick self-assessment: do I need calibration?

Before deciding whether to apply calibration, consider the following questions:

- *Do probability values drive decisions?* If only the ranking matters (e.g., a top-k recommendation list), calibration may be unnecessary.
- *Are probabilities communicated to users?* If yes, they must be interpretable as real frequencies.
- *Do I have sufficient held-out data?* Isotonic regression needs thousands of examples; Platt scaling and temperature scaling can work with a few hundred.
- *Are there subgroups whose calibration should be checked separately?* Global calibration does not guarantee per-group calibration.

8 Summary

Concept	**Key idea**
Model calibration	Procedure to adjust predicted probabilities so that a predicted probability q matches the observed fraction of positives among instances assigned that score
Calibration vs. discrimination	A model can have high AUC (good ranking) but poor calibration (untrustworthy probability values); the two properties are orthogonal and require separate metrics
Calibration curve	Visual tool plotting mean confidence per bin against observed fraction of positives per bin; perfect calibration lies on the diagonal $y = x$
Overconfidence	$\text{conf}(B_m) > \text{acc}(B_m)$: the model overestimates probability; curve lies *below* the diagonal
Underconfidence	$\text{conf}(B_m) < \text{acc}(B_m)$: the model underestimates probability; curve lies *above* the diagonal
ECE	Weighted average of per-bin *absolute* calibration errors; depends on binning strategy and is not comparable across different binning choices; lower is better
Brier Score	Mean squared error between predicted probabilities and true labels; strictly proper scoring rule; decomposes into reliability (uses *squared* error, distinct from ECE), resolution (discrimination), and uncertainty (task difficulty); higher resolution *reduces* the score
Log Loss (NLL)	Cross-entropy of predictions against true labels; strictly proper; penalizes overconfident wrong predictions very heavily
Platt Scaling	Logistic regression on base model scores; parametric; works well for sigmoid-shaped miscalibration; requires few calibration examples
Isotonic Regression	Non-parametric monotone function fit; more flexible than Platt scaling but requires more calibration data; may overfit with small calibration sets and may introduce ties that slightly *reduce* AUC-ROC
Temperature Scaling	Single scalar T applied to logits before sigmoid (binary) or softmax (multiclass); popular for neural networks; does not change predictions, only confidences
Calibration patterns	Decision Trees (unpruned): overconfident, cluster near 0 and 1. Random Forests: tend to be *underconfident* due to probability averaging [Niculescu-Mizil and Caruana, 2005]. Deep neural networks: typically overconfident.
AUC preservation	Platt Scaling and Temperature Scaling are strictly monotone $\Rightarrow$ AUC-ROC unchanged *exactly*. Isotonic Regression may introduce ties $\Rightarrow$ minor AUC-ROC reduction possible.

Key Takeaways

1. **Calibration and discrimination are different**: a model with high AUC-ROC can still be badly miscalibrated; always assess both.
2. **The calibration curve is the primary diagnostic**: plot it before and after calibration. Points *below* the diagonal mean the model overestimates probability (overconfident), and points *above* mean it underestimates probability (underconfident).
3. **ECE is the standard calibration metric**: it quantifies the average *absolute* gap between confidence and observed positive rate; always report the binning strategy alongside it, as ECE values are not comparable across different binning choices.
4. **Calibration requires a separate, held-out calibration set**: using training data to fit the calibration model introduces data leakage and produces overoptimistic results that do not generalize.
5. **Choose the calibration method based on dataset size and model type**: Platt scaling for small calibration sets or simple models; isotonic regression when data is plentiful and miscalibration is complex (but beware of overfitting with small calibration sets); temperature scaling for neural networks (binary: sigmoid; multiclass: softmax).
6. **Calibration preserves discrimination differently by method**: Platt scaling and temperature scaling are strictly monotone and therefore leave AUC-ROC *exactly* unchanged; isotonic regression may introduce ties that cause minor reductions.
7. **Random Forests tend to be underconfident, not overconfident**: averaging across many trees compresses probabilities toward 0.5; do not confuse this with the behaviour of individual unpruned Decision Trees.
8. **Calibration matters most when probabilities drive decisions**: threshold selection, expected utility, risk communication, and fairness auditing all depend on the probability values, not just their ordering.

8 References

Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 1321–1330. PMLR, 2017.

Allan H. Murphy. A new vector partition of the probability score. *Journal of Applied Meteorology*, 12(4):595–600, 1973.

Alexandru Niculescu-Mizil and Rich Caruana. Predicting good probabilities with supervised learning. In *Proceedings of the 22nd International Conference on Machine Learning*, ICML, pages 625–632, 2005.

John C. Platt. Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. In *Advances in Large Margin Classifiers*, pages 61–74. MIT Press, 1999.

Bianca Zadrozny and Charles Elkan. Transforming classifier scores into accurate multiclass probability estimates. In *Proceedings of the 8th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 694–699, 2002.